# SIMATS ENGINEERING

## Department of Computer Science & Engineering

### ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

| | | | |
|---|---|---|---|
| **Course Code:** | **CSA12** | **Course Title:** | **Computer Architecture for Multicore Systems** |
| **CO Assessed:** | **CO5** | **Assessment:** | **ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)** |
| **Weightage:** | **30%** | **Total Marks:** | **25** |
| **CO5 Statement:** | *Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.* | | |
| **Student Name:** | KAMALESH K | **Reg.No.:** | 192572164 |
| **Department:** | B TECH CSE(AI) | **Date:** | 01\09\2026 |

## ANNEXURE A

### Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

***Code of Conduct:***

*I KAMALESH K(192572164) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:KAMALESH K***

## MARKS ALLOCATION

| | Problem Understanding (20%) | Algorithm Design (30%) | Use of Control Structures (20%) | Pseudocode Structure (10%) | Completeness (20%) |
|---|---|---|---|---|---|
| **Q1** | | | | | |
| **Q2** | | | | | |
| **Q3** | | | | | |
| **Q4** | | | | | |
| **Q5** | | | | | |

## RUBRICS FOR CALCULATION:

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Problem Understanding | 20% | Identifies all inputs, outputs, constraints accurately | Minor missing elements | Partial understanding | Incorrect interpretation |
| Algorithm Design | 30% | Complete, correct, and logically sound solution | Minor logical gaps | Partially correct logic | Incorrect approach |
| Use of Control Structures | 20% | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies | Limited or basic usage | Incorrect or missing constructs |
| Pseudocode Structure | 10% | Well-structured, clear, consistent format, easy to follow | Mostly clear with minor issues | Difficult to follow in parts | Poorly structured and unclear |
| Completeness | 20% | Handles all cases; optimized approach | Handles most cases; reasonable efficiency | Handles basic cases only | Incomplete or inefficient |
| | | | | | |

# 1.DMA-Based Data Transfer

```
START
Initialize DMA Controller
Set Source = I/O Device
Set Destination = Memory
Set Data_Size
Enable DMA Interrupt

Request DMA Transfer

WHILE Transfer_Not_Complete
   CPU performs other tasks
END WHILE

On DMA Interrupt:
   Check Transfer Status
   IF Transfer_Successful
      Display "Transfer Complete"
   ELSE
      Display "Transfer Error"
   END IF

STOP
```

# 2. Prioritized Interrupt Handling

START

Initialize Interrupt Controller

WHILE System is Running

  Check Interrupt Requests

  IF Interrupt Exists

    Find Highest Priority Interrupt

    Identify Device

    Save CPU State

    Call Corresponding ISR

    Clear Interrupt

    Restore CPU State

  END IF

END WHILE

STOP

# 3. Dynamic I/O Selection

START

Read Latency Requirement

Read Bandwidth Requirement

IF Latency is Low AND Bandwidth is High

  Select Memory-Mapped I/O

ELSE IF Latency is High OR Bandwidth is Low

  Select I/O-Mapped I/O

ELSE

  Compare Both Methods

  Select Better Method

END IF

Configure Selected I/O Method

START Data Transfer

STOP

# 4. Bus Arbitration for PCIe, USB and Thunderbolt

START

Initialize Bus Controller

WHILE Devices Request Bus

Receive Requests from PCIe, USB, Thunderbolt

Assign Priority to Each Request

Select Highest Priority Device

Grant Bus to Selected Device

Allow Data Transfer

Release Bus

END WHILE

STOP

# 5. Hybrid I/O Communication

```
START
Initialize I/O Devices

FOR Each Device
  IF Device is Low-Speed
    Use Polling
    Check Device Status
    Transfer Data
  ELSE
    Use DMA
    Enable Interrupt
    Start Data Transfer

    WHEN Transfer Complete
      Generate Interrupt
      Handle Interrupt
    END WHEN
  END IF
END FOR

STOP
```